

Bishop's University

Department of Computer Science

Programming Languages for DA - CS504

Assignement 2 - FALL2023

To do:

- Provide the equivalent R code of the entire work.

load data

```
In [208... import pandas as pd
df = pd.read_csv('Data/csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_confirmed_global.csv')
```

```
In [209... df.head(2)
```

```
Out[209]:
```

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	3/24/20	3/25/20	3/26/20	3/27/20	3/28/20
0	NaN	Afghanistan	33.0000	65.0000	0	0	0	0	0	0	...	74	84	94	110	110
1	NaN	Albania	41.1533	20.1683	0	0	0	0	0	0	...	123	146	174	186	186

2 rows × 76 columns

Get only China's data

Data shows daily accumulation of confirmed cases from 01/22/2020 to 04/02/2020

```
In [210... groups = df.groupby('Country/Region')
China = groups.get_group('China')
China.head(2)
```

```
Out[210]:
```

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	3/24/20	3/25/20	3/26/20	3/27/20	3/28/20
49	Anhui	China	31.8257	117.2264	1	9	15	39	60	70	...	990	990	990	990	990
50	Beijing	China	40.1824	116.4142	14	22	36	41	68	80	...	558	561	566	569	569

2 rows × 76 columns

Discard Country/region, Lat and Long columns

```
In [211... China_2 = China.drop(['Country/Region', 'Lat', 'Long'], axis=1)
China_2.head()
```

```
Out[211]:
```

	Province/State	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	1/28/20	1/29/20	1/30/20	...	3/24/20	3/25/20	3/26/20	3/27/20	3/28/20
49	Anhui	1	9	15	39	60	70	106	152	200	...	990	990	990	990	990
50	Beijing	14	22	36	41	68	80	91	111	114	...	558	561	566	569	573
51	Chongqing	6	9	27	57	75	110	132	147	182	...	578	578	578	578	578
52	Fujian	1	5	10	18	35	59	80	84	101	...	318	322	328	331	337
53	Gansu	0	2	2	4	7	14	19	24	26	...	136	136	136	136	136

5 rows × 73 columns

Compute daily confirmed cases by provinces

```
In [232... # get only the values of cases
# from day 1 to one before last day
X1 = China_2.iloc[:,1:-1].values
# from day 2 to last day
X2 = China_2.iloc[:,2:].values
# compute the difference (case_day_i - cases_day_(i-1))
daily_cases = X2-X1
print(daily_cases)
```

```
[[ 8  6 24 ...  0  0  0]
 [ 8 14  5 ...  3  0  2]
 [ 3 18 30 ...  0  0  0]
 ...
 [ 2  0  1 ...  0  0  0]
 [ 1  3  6 ...  2  0  1]
 [17 16 19 ...  2  0  1]]
```

```
In [214... # Save the daily cases (daily_cases) in a new dataframe
col_names = China_2.columns
provinces = col_names[0]
days = col_names[1:]
day_1 = days[0]
other_days = days[1:]
```

```
In [215... China_dc = pd.DataFrame()
China_dc['Province'] = China_2[provinces]
China_dc[day_1] = China_2[day_1]
China_dc[other_days] = daily_cases
China_dc.head(3)
```

```
Out[215]:
```

	Province	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	1/28/20	1/29/20	1/30/20	...	3/24/20	3/25/20	3/26/20	3/27/20	3/28/20	3/29/20
49	Anhui	1	8	6	24	21	10	36	46	48	...	0	0	0	0	0	0
50	Beijing	14	8	14	5	27	12	11	20	3	...	21	3	5	3	4	4
51	Chongqing	6	3	18	30	18	35	22	15	35	...	0	0	0	0	0	0

3 rows × 73 columns

Reset index to start from 0

```
In [216... China_dc = China_dc.reset_index(drop=True)
China_dc.head()
```

```
Out[216]:
```

	Province	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	1/28/20	1/29/20	1/30/20	...	3/24/20	3/25/20	3/26/20	3/27/20	3/28/20	3/29/20
0	Anhui	1	8	6	24	21	10	36	46	48	...	0	0	0	0	0	0
1	Beijing	14	8	14	5	27	12	11	20	3	...	21	3	5	3	4	4
2	Chongqing	6	3	18	30	18	35	22	15	35	...	0	0	0	0	0	0
3	Fujian	1	4	5	8	17	24	21	4	17	...	5	4	6	3	6	6
4	Gansu	0	2	0	2	3	7	5	5	2	...	0	0	0	0	0	0

5 rows × 73 columns

Province as columns and days as rows

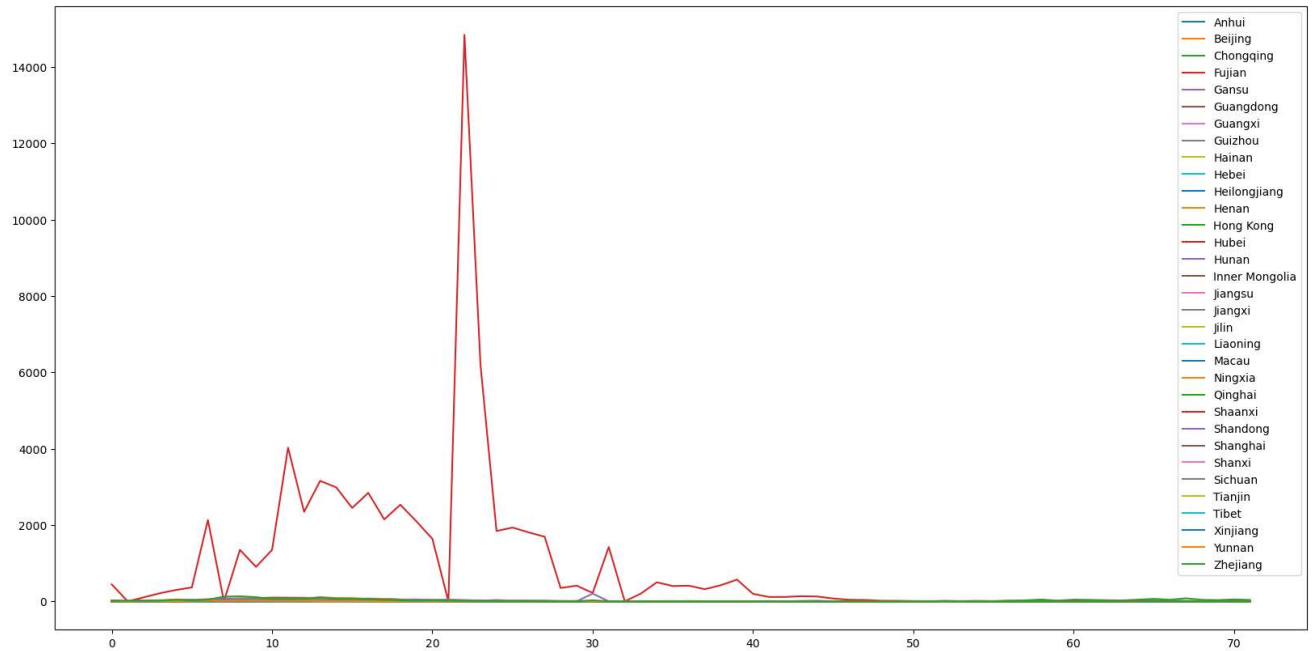
```
In [217... China_dc_val = China_dc.values
Province_names = China_dc_val[:,0]
Cases = China_dc_val[:,1:].T
#----- make a pandas frame -----
China_dc_t = pd.DataFrame(Cases, columns = Province_names)
China_dc_t.head()
```

Out[217]:	Anhui	Beijing	Chongqing	Fujian	Gansu	Guangdong	Guangxi	Guizhou	Hainan	Hebei	...	Shaanxi	Shandong	Shanghai	Shanxi	Sichuan	Tianjin
0	1	14	6	1	0	26	2	1	4	1	...	0	2	9	1	5	4
1	8	8	3	4	2	6	3	2	1	0	...	3	4	7	0	3	0
2	6	14	18	5	0	21	18	0	3	1	...	2	9	4	0	7	4
3	24	5	30	8	2	25	0	1	11	6	...	10	12	13	5	13	2
4	21	27	18	17	3	33	13	1	3	5	...	7	19	7	3	16	4

5 rows × 33 columns

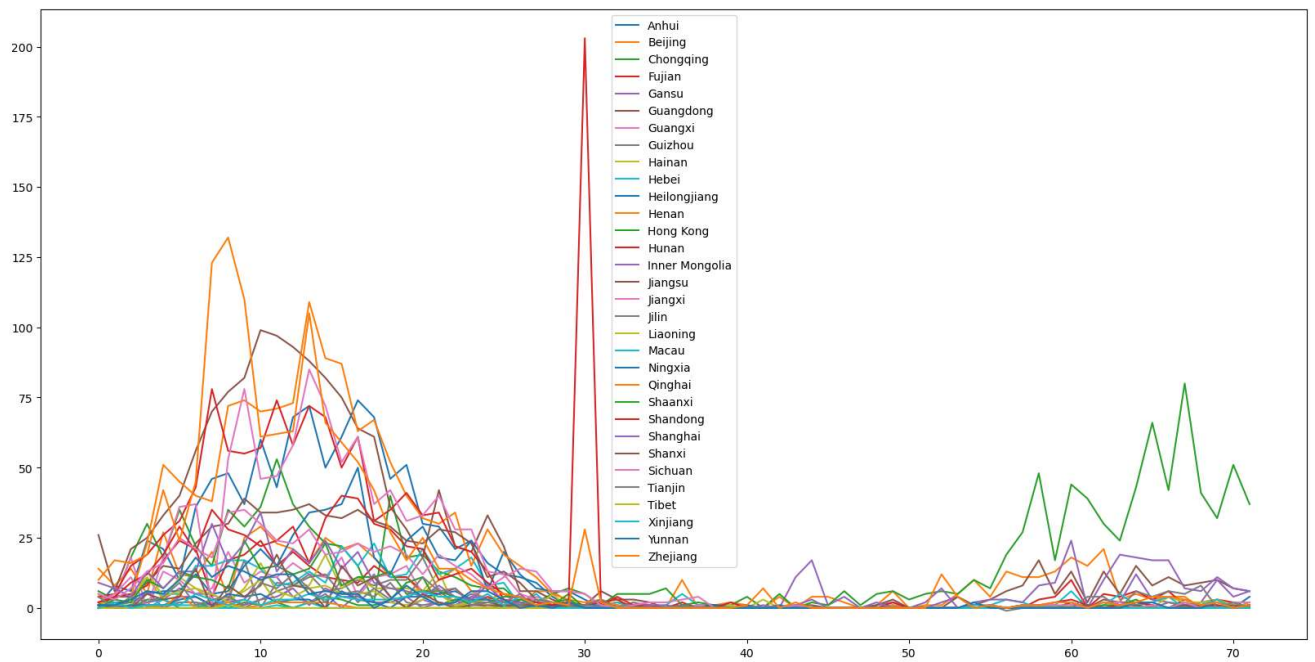
Display daily confirmed cases

```
In [218... import matplotlib.pyplot as plt
China_dc_t.plot(figsize=(20,10))
plt.show()
```



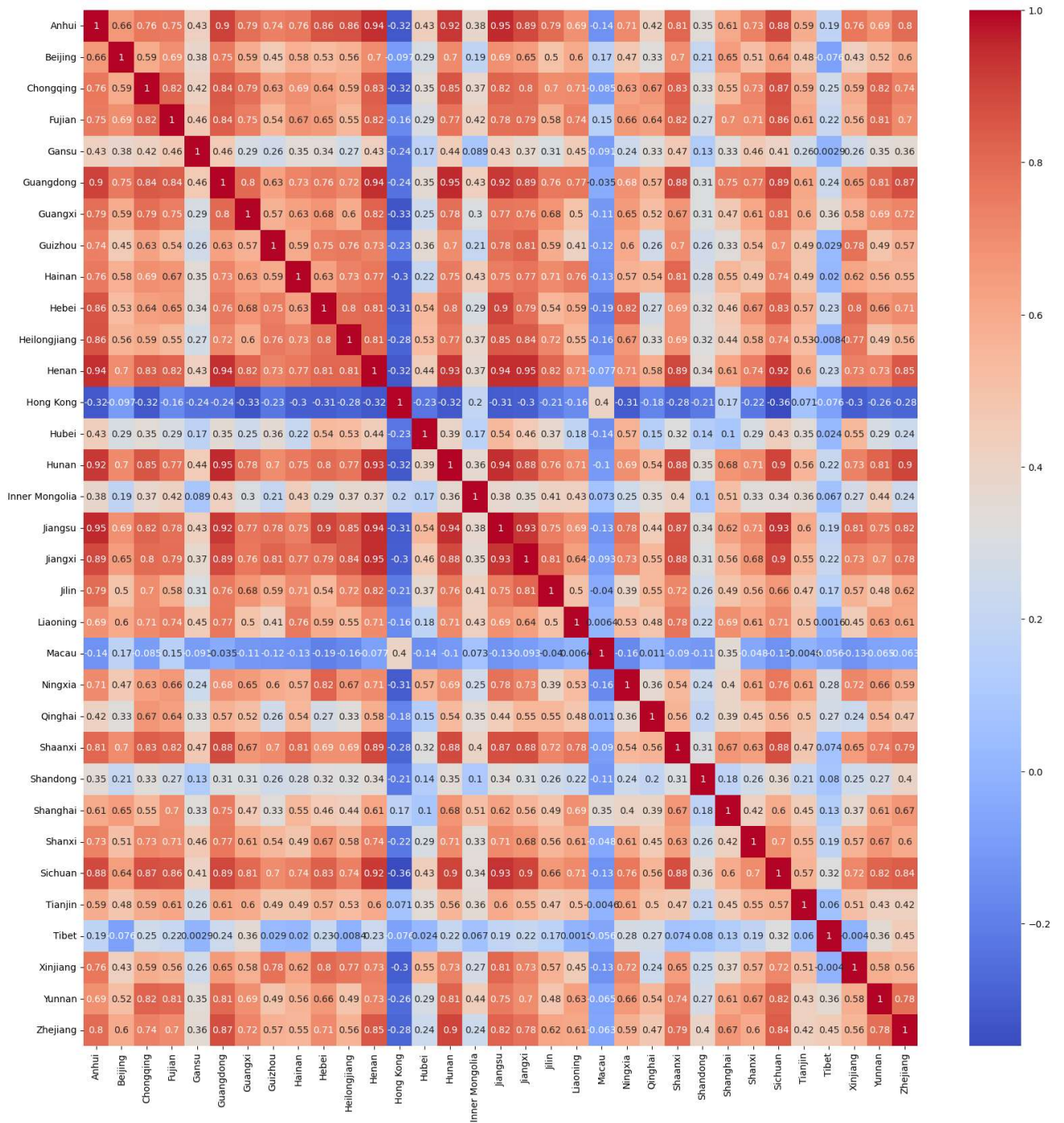
As shown Hubei province is a special case. Therefore, discard it from the data

```
In [221... # Discard Hubei province. It has huge number of cases
China_4 = China_dc_t.drop('Hubei', axis=1)
China_4.plot(figsize=(20,10))
plt.show()
```



Correlation between provinces in terms of daily confirmed cases

```
In [196... # Correlation
#Using Pearson Correlation
import seaborn as sns
plt.figure(figsize=(20, 20))
sns.heatmap(China_dc_t.corr(numeric_only=False), annot=True, cmap='coolwarm')
plt.show()
```



Sort the correlations

```
In [222... correlation = China_dc_t.corr(numeric_only=False)
c = correlation.abs().unstack()
c.sort_values(ascending = False)[33:64][::2]
```

```
Out[222]:
Anhui      Jiangsu      0.948884
Hunan      Guangdong    0.947127
Henan      Jiangxi      0.946075
Hunan      Jiangsu      0.943732
Jiangsu     Henan        0.941800
Guangdong   Henan        0.939586
Anhui      Henan        0.936565
Sichuan     Jiangsu      0.932564
Henan      Hunan        0.929992
Jiangxi     Jiangsu      0.929468
Hunan      Anhui        0.922236
Guangdong   Jiangsu      0.920927
Henan      Sichuan      0.919194
Guangdong   Anhui        0.902344
Hebei       Jiangsu      0.897629
Hunan      Zhejiang     0.896465
dtype: float64
```

PCA

- Assume the days are the variables and the provinces are the instances
- Apply PCA and show the scatter of the first 3 PC.

```
In [224... # Get the values
X = China_dc[days].values
print(X)

[[ 1  8  6 ...  0  0  0]
 [14  8 14 ...  3  0  2]
 [ 6  3 18 ...  0  0  0]
 ...
 [ 0  2  0 ...  0  0  0]
 [ 1  1  3 ...  2  0  1]
 [10 17 16 ...  2  0  1]]
```

```
In [225... # PCA
from sklearn.decomposition import PCA
pca = PCA(n_components=3)

pca.fit(X)
# transform data
X_reduced = pca.transform(X)
X_reduced.shape
```

```
Out[225]: (33, 3)
```

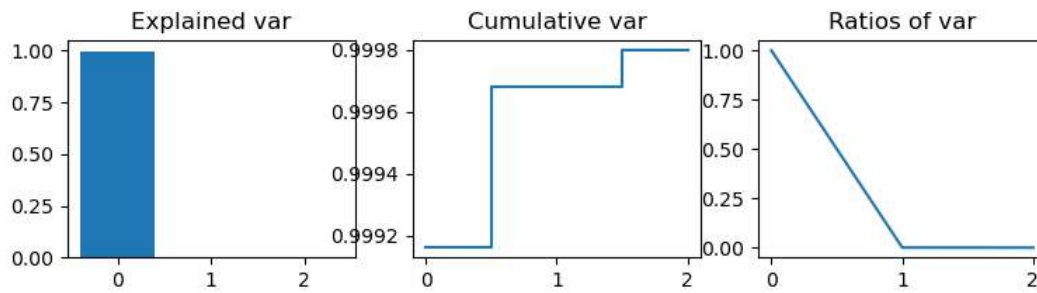
Display Explained variances, cumulative variances and the respecting ratios

```
In [226... exp_var = pca.explained_variance_ratio_
cumul_var = np.cumsum(exp_var)
exp_var_ratio = pca.explained_variance_ratio_
```

```
In [227... print('Explained variances: ', exp_var)
print('Cumulative variances: ', cumul_var)
print('Ratio explained variances: ', exp_var_ratio)
```

```
Explained variances: [9.99161637e-01 5.16487694e-04 1.18391188e-04]
Cumulative variances: [0.99916164 0.99967812 0.99979652]
Ratio explained variances: [9.99161637e-01 5.16487694e-04 1.18391188e-04]
```

```
In [228... plt.figure(figsize=(9,2))
plt.subplot(1,3,1)
plt.bar(range(0,len(exp_var)), exp_var)
plt.title('Explained var')
plt.subplot(1,3,2)
plt.step(range(0,len(exp_var)), cumul_var, where='mid')
plt.title('Cumulative var')
plt.subplot(1,3,3)
plt.plot(exp_var_ratio)
plt.title('Ratios of var')
plt.show()
```

Save the PC vales in a DataFrame

```
In [229]: # save X_s in a dataframe
X_reduced_df = pd.DataFrame(data=X_reduced)
X_reduced_df.columns = ['w1', 'w2', 'w3']
X_reduced_df.head(5)
```

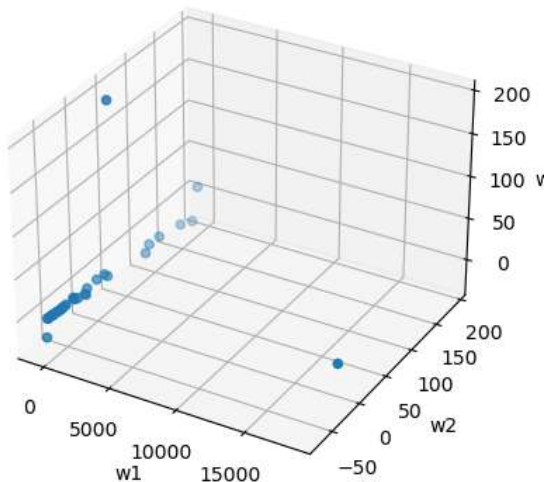
```
Out[229]:
```

	w1	w2	w3
0	-496.977429	101.301488	-11.812384
1	-572.313074	-8.321902	-8.183773
2	-556.370960	24.567849	-4.004067
3	-586.911842	-19.065950	-6.718658
4	-607.252647	-53.302153	-2.534793

Display the scatter of data using 1st three PC

```
In [230]: # 3D plot
plot3d = plt.axes (projection='3d')
x = X_reduced_df['w1'].values
y = X_reduced_df['w2'].values
z = X_reduced_df['w3'].values
plot3d.scatter3D (x, y, z)
plot3d.set_xlabel ('w1')
plot3d.set_ylabel ('w2')
plot3d.set_zlabel ('w3')
```

```
Out[230]: Text(0.5, 0, 'w3')
```



Plot the scatter of data using two of the PC

```
In [231]: # Plot histogram
sns.scatterplot(data=X_reduced_df, x="w2", y="w3")
# pandas
#X_df.plot.scatter(x='Por', y='TOC')
plt.show()
```

